

Manuale di Installazione

Programmazione Web, Il progetto: Web Services (Caso B)

by Archonet

1. Introduzione

Questo progetto migra i dati del database del Primo Progetto (AcquaFlow) verso un nuovo database PostgreSQL locale.

La migrazione avviene tramite tre componenti collegati: un web-service PHP remoto (fornisce i dati), una Servlet Java intermedia (trasferisce i dati), un web-service Django locale (riceve e salva i dati).

Il presente documento descrive i passi necessari per installare ed eseguire il sistema.

2. Prerequisiti

Installare sulla macchina il seguente software ed eseguire i passaggi elencati:

- Python, versione 3.12 o successiva
- Java Development Kit (JDK), versione 17 o successiva
- PostgreSQL, qualsiasi versione recente
- Impostare una password per l'utente amministratore di PostgreSQL, necessaria per i passaggi successivi:

Su *Linux*, l'utente amministrativo si chiama "postgres". Impostare una password, una sola volta, con privilegi di amministratore:

```
sudo -u postgres psql -c "ALTER USER postgres PASSWORD  
'SCEGLI_UNA_PASSWORD';"
```

Su *Mac*, l'utente amministrativo spesso corrisponde al proprio nome utente del computer, non a "postgres". Impostare una password per il proprio utente:

```
psql postgres -c "ALTER USER $(whoami) PASSWORD  
'SCEGLI_UNA_PASSWORD';"
```

(il comando racchiuso in \$() inserisce automaticamente il proprio nome utente)

(questa password, e su *Mac* anche il nome utente, verranno richiesti più avanti.)

- Git
- Apache Tomcat, versione 11 (o successiva compatibile con Jakarta Servlet 6.1)
- Connessione a Internet attiva (necessaria per raggiungere il web-service PHP remoto e, al primo avvio, per lo scaricamento automatico di Maven)
- **Non** installare Maven separatamente: il progetto include un Maven Wrapper, che lo scarica automaticamente al primo utilizzo. Non è richiesto alcun IDE.
- Impostazione di JAVA_HOME e del PATH (*Windows*):

Questo passaggio riguarda solo *Windows*: su *Linux* e *Mac*, gli strumenti installati tramite gestore di pacchetti (es. apt, Homebrew) vengono collocati automaticamente in percorsi già inclusi nel PATH, senza bisogno di configurazione aggiuntiva.

Ci sono 2 vie, Manuale o Automatica:

➤ Manuale:

Aggiungere PostgreSQL al PATH:

1. Individuare la cartella bin di PostgreSQL (tipicamente C:\Program Files\PostgreSQL[versione]\bin)

2. Premere il tasto Windows, digitare "variabili d'ambiente", aprire "Modifica le variabili di ambiente relative al sistema"
3. Cliccare "Variabili d'ambiente...", nella sezione "Variabili utente" selezionare la riga Path e cliccare "Modifica..."
4. Cliccare "Nuovo" e incollare il percorso trovato al punto 1
5. Confermare con OK su tutte le finestre, chiudere e riaprire il terminale

Impostare JAVA_HOME:

1. Da terminale, digitare `where.exe java` per trovare il percorso di installazione, poi togliere `%bin%java.exe` dalla fine
2. Ripetere il punto 2 della procedura sopra (fino ad aprire la finestra "Variabili d'ambiente..."), poi nella sezione "Variabili utente" cliccare direttamente "Nuovo" per creare una variabile chiamata JAVA_HOME con quel percorso come valore
3. Confermare, chiudere e riaprire il terminale

➤ **Automatica:**

In alternativa a impostare `JAVA_HOME` e il resto del `PATH` a mano, è disponibile uno script che individua automaticamente Java, PostgreSQL e Python già installati sul computer, senza bisogno di configurare nulla manualmente.

Attenzione: Va eseguito una volta ad ogni nuova finestra di terminale aperta, con questa sintassi esatta (il punto iniziale, seguito da uno spazio, è necessario: senza di esso lo script non ha alcun effetto):

```
. .\setup_env.ps1
```

Alternativa: Installazione rapida dei prerequisiti (*Linux*):

Su *Linux (Debian/Ubuntu)*, questo comando installa/aggiorna la maggior parte dei prerequisiti (Tomcat escluso):

```
sudo apt update && sudo apt install -y postgresql postgresql-contrib python3 python3-venv python3-pip python3-dev openjdk-17-jdk-lsof curl libpq-dev build-essential
```

Su *Windows* non esiste un comando equivalente universale: verificare/aggiornare ciascun prerequisito individualmente.

3. Struttura del Progetto

- db/ *schema del database PostgreSQL*
- django/ *web-service locale (Django)*
- servlet/ *web-service intermedio (Servlet Java)*
- php/ *copia documentale del web-service remoto (in esecuzione online, non richiede installazione locale)*
- docs/ *documentazione*

4. Installazione

0) **Preparare una cartella di lavoro e aprire il terminale**

Creare una nuova cartella vuota sul proprio computer per ospitare tutto il progetto (il nome non è importante).

Aprire un [terminale](#) (programma che permette di eseguire comandi scrivendoli) posizionato esattamente dentro questa cartella:

Su Windows:

aprire la cartella appena creata in Esplora File. Fare clic nella barra dell'indirizzo in alto (dove appare il percorso), cancellare il testo presente, digitare `powershell` e premere Invio. Si apre un terminale già posizionato nella cartella corretta.

Su Mac:

aprire l'applicazione Terminale (in Applicazioni > Utility). Digitare `cd` seguito da uno `spazio`, poi trascinare l'icona della cartella appena creata dentro la finestra del terminale: il percorso viene inserito automaticamente. Premere Invio.

1) Clonare la repository

Nel terminale aperto al Passo 0, digitare:

```
git clone https://github.com/FedericoCremonesiUniBG/acquaflow2.git
cd acquaflow2
```

Ci sono 2 vie per proseguire con l'installazione; Automatica o Manuale:

➤ Automatica = utilizzare uno script automatico:

È disponibile uno script che automatizza l'intera installazione descritta in questo manuale (`install.ps1` per *Windows*, `install.sh` per *Mac/Linux*), riducendola a un solo comando. Dalla cartella principale del progetto (in cui ci si trova già, grazie a `cd acquaflow2`):

Su Windows:

```
.\install.ps1 -TomcatPath "<percorso-tomcat>"
```

Su Mac/Linux:

```
chmod +x install.sh
./install.sh <percorso-tomcat>
```

➤ Manuale = seguire manualmente i passaggi 2) -> 9):

Proseguire qui sotto:

2) Creare un utente dedicato e il database PostgreSQL

Questo passo crea un utente specifico per il progetto, invece di usare l'utente amministratore generale di PostgreSQL.

Su Mac, se l'utente amministratore non si chiama "postgres" (vedi nota nei Prerequisiti), sostituire "postgres" con il proprio nome utente in tutti i comandi seguenti.

- Creare il nuovo utente dedicato (sostituire SCEGLI_UNA_PASSWORD con una password a scelta e annotarla, poiché servirà più volte in futuro):

```
psql -h localhost -U postgres -c "CREATE USER acquaflow_app WITH
PASSWORD 'SCEGLI_UNA_PASSWORD';"
```

- Creare il database, assegnandolo al nuovo utente come proprietario:

```
psql -h localhost -U postgres -c "CREATE DATABASE acquaflow_locale
OWNER acquaflow_app;"
```

- Dare al nuovo utente il permesso di creare tabelle (necessario dalla versione 15 di PostgreSQL in poi, che per sicurezza non lo concede più automaticamente):

```
psql -h localhost -U postgres -d acquaflow_locale -c "GRANT ALL ON
SCHEMA public TO acquaflow_app;"
```

- Rendere l'utente proprietario dello schema, per una compatibilità più solida su alcune installazioni:

```
psql -h localhost -U postgres -d acquaflow_locale -c "ALTER SCHEMA public OWNER TO acquaflow_app;"
```

- Applicare lo schema, connettendosi questa volta con il nuovo utente (verrà richiesta la password appena scelta):

```
psql -h localhost -U acquaflow_app -d acquaflow_locale -f db/schema_postgres.sql
```

3) Preparare l'ambiente Python

Un "ambiente virtuale" è una copia isolata dei programmi Python necessari a questo progetto, separata dal resto del computer. Evita conflitti con altri programmi eventualmente già installati.

```
cd django
```

```
python -m venv venv
```

Attivare l'ambiente virtuale. Da questo momento, e finché non si chiude il terminale, i comandi Python useranno questa copia isolata invece di quella generale del computer.

Su Windows (PowerShell):

```
.\venv\Scripts\Activate.ps1
```

Se questo comando restituisce un errore relativo a "esecuzione degli script disabilitata", eseguire prima quest'altro comando (una sola volta, non va ripetuto le volte successive), poi riprovare il comando sopra:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Su Mac/Linux:

```
source venv/bin/activate
```

Se l'attivazione è riuscita, il terminale mostra la scritta (venv) all'inizio della riga, prima del cursore.

Installare le dipendenze (programmi Python necessari):

```
pip install -r requirements.txt
```

4) Configurare le credenziali del database

Questo file dice al programma come collegarsi al database creato al Passo 2: nome, utente, password.

Nella cartella `django`, individuare il file chiamato `.env.example`. Farne una copia e rinominare la copia in `.env` (attenzione: il nome inizia con un punto, e non ha altre estensioni).

Aprire `.env` con un editor di testo semplice (es. su Windows: `Blocco Note`) e inserire questi valori, sostituendo `INSERISCI_PASSWORD` con la password scelta per l'utente `acquaflow_app` al Passo 2:

```
DB_NAME=acquaflow_locale
```

```
DB_USER=acquaflow_app
```

```
DB_PASSWORD=INSERISCI_PASSWORD
```

```
DB_HOST=localhost
```

```
DB_PORT=5432
```

Salvare e chiudere il file.

5) Applicare le migrazioni Django

Questo comando prepara Django per lavorare con il database creato al Passo 2.

```
python manage.py migrate
```

6) Avviare il web-service Django

```
python manage.py runserver
```

Se l'avvio ha successo, tra le ultime righe mostrate compare la scritta `Starting development server at http://127.0.0.1:8000/`. In caso contrario, leggere il messaggio d'errore mostrato: indica solitamente cosa non ha funzionato nei passi precedenti.

Lasciare questo terminale aperto per tutta la durata dell'utilizzo: il servizio deve restare in ascolto su <http://127.0.0.1:8000>.

7) Compilare la Servlet

Aprire un nuovo terminale, separato e aggiuntivo rispetto a quello del Passo 6 (che deve restare aperto).

Ripetere la stessa procedura del Passo 0 per posizionarlo nella cartella principale del progetto. Poi:

```
cd servlet
```

Su Windows:

```
.\mvnw.cmd clean package
```

Su Mac/Linux:

```
chmod +x mvnw
```

```
./mvnw clean package
```

Al primo avvio, il comando scarica Maven automaticamente: non richiede installazione manuale. Al termine, il file generato si trova in [target/migrazione.war](#).

8) Distribuire la Servlet su Tomcat

Aprire Esplora File (o Finder su Mac). Nella cartella del progetto (quella in cui ci si è posizionati con il terminale), andare nella sottocartella [servlet](#), poi [target](#): qui si trova il file [migrazione.war](#), generato al passo precedente. Copiarlo e incollarlo dentro la cartella [webapps](#), che si trova nella cartella in cui è stato installato Tomcat.

Avviare Tomcat. Spostarsi anzitutto con il terminale (aprirne uno nuovo va bene) dentro la cartella bin dell'installazione di Tomcat (sostituire interamente `<percorso-tomcat>`, incluse le parentesi `<` e `>`, che non vanno digitate, con il percorso reale della cartella dove è installato Tomcat sul proprio computer), poi avviarlo da lì:

Su Windows:

```
cd <percorso-tomcat>\bin
```

```
.\startup.bat
```

Su Mac/Linux:

```
cd <percorso-tomcat>/bin
```

```
./startup.sh
```

9) Avviare la migrazione

Aprire un browser web. Visitare il seguente indirizzo:

<http://localhost:8080/migrazione/migra>

Attendere la risposta. La pagina mostra il numero di record migrati per ciascuna tabella. L'operazione può richiedere alcuni minuti, a causa del volume di dati nella tabella delle letture.

5. Verifica dell'installazione

Per confermare che i dati siano stati migrati correttamente, eseguire lo script di verifica incluso nel progetto. Posizionarsi nella cartella principale del progetto (rieseguendo il Passo 0 e `cd acquaflow2`) ed eseguire:

Su Windows:

```
.\verifica.ps1
```

Su Mac/Linux:

```
chmod +x verifica.sh  
./verifica.sh
```

Lo script confronta, tabella per tabella, il numero di record presenti nel database locale con quello della fonte originale, stampando "corrispondono" o segnalando eventuali differenze.

6. Arresto dei Servizi

Al termine dell'utilizzo, fermare Django con la combinazione di tasti Ctrl+C nel terminale corrispondente.

Fermare Tomcat spostandosi anzitutto con il terminale nella cartella bin dell'installazione:

Su Windows:

```
cd <percorso-tomcat>\bin  
.\shutdown.bat
```

Su Mac/Linux:

```
cd <percorso-tomcat>/bin  
./shutdown.sh
```

7. Risoluzione dei Problemi

- **Il comando `psql` non viene riconosciuto**
La cartella `bin` dell'installazione di PostgreSQL non è inclusa nel PATH di sistema. Aggiungere il percorso alle variabili d'ambiente e aprire un nuovo terminale.
- **Errore "`ModuleNotFoundError: No module named 'django'`"**
L'ambiente virtuale Python non è attivo nel terminale corrente. Ripetere l'attivazione descritta al Passo 3.
- **Errore di connessione durante l'avvio della migrazione**
Verificare che sia il web-service Django (Passo 6), sia Tomcat (Passo 8) siano entrambi avviati e in esecuzione contemporaneamente, in due terminali separati.
- **Il comando `mvnw.cmd` (o `./mvnw`) restituisce un errore relativo a Java**
Verificare che la variabile d'ambiente `JAVA_HOME` sia impostata e punti alla cartella di installazione del JDK.
- **La porta 8000 o 8080 risulta già occupata**
Un'altra istanza di Django o Tomcat è già in esecuzione. Chiuderla prima di riavviare il servizio.
- **Il file `.env` perde il punto iniziale durante la rinomina (*Windows*)**

Rinominare di nuovo il file, assicurandosi di scrivere .env con il punto come primo carattere. Se Esplora File nasconde le estensioni dei file, il punto potrebbe non essere visibile correttamente: verificare rinominando una seconda volta.

- **Errore "role postgres does not exist" (Mac)**

Alcune installazioni di PostgreSQL su macOS non creano l'utente amministratore "postgres", usando invece il nome dell'account del computer. Creare il ruolo mancante con il comando: `createuser -s postgres`.